

F O R T I M A N A G E R N S E 6

Lab 06 — Scripts, Templates and Metadata Variables

01. Scripts

Everything in Labs 4 and 5 was done one device at a time through the interface. Scripts are the first mechanism in FortiManager that scales past that — a block of FortiGate CLI stored once and run against one device, many devices, or a device group.

The lab continues on the pair from Lab 5: FortiManager 7.4.3 and FortiGate 7.2.1, now carrying the hostname fortigate7-2-noooo after the local changes made there.

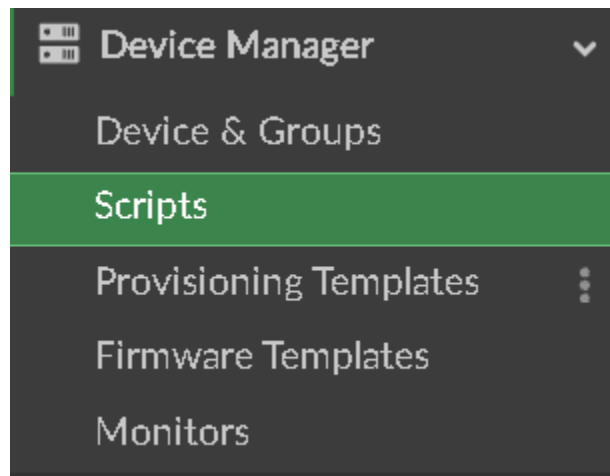


Figure 1 — Device Manager > Scripts.

► 1.1 Creating a script

A script carries a name, a type, and a target. The script used here creates a firewall address object.

```
config firewall address
  edit "Lab-Script-Object"
    set subnet 10.99.99.0 255.255.255.0
  next
end
```

Figure 2 — Create New Script — note the Run script on field, set to Remote FortiGate Directly (via CLI).

The **Run script on** field decides whether the script reaches the firewall immediately or only reaches FortiManager's copy of it. The three targets behave differently enough that choosing the wrong one changes what the script means.

Target	What it touches
Device Database	FortiManager's stored copy of the device. Nothing reaches the firewall until an install is run.
Remote FortiGate Directly (via CLI)	The live firewall, immediately, over the FGFM tunnel. The device database is not updated by the script itself.
Policy Package or ADOM Database	ADOM-level objects and policy packages rather than a single device.

► 1.2 Running against the remote FortiGate



Figure 3 — Run Script on Device — the script targeting the remote FortiGate.

The object exists on the firewall as soon as the script finishes, and the configuration status in FortiManager does not move.



Figure 4 — Lab-Script-Object present in the FortiGate address list.

► 1.3 Running against the device database

A second script was written to create a different object, with the target changed to Device Database.

```
config firewall address
  edit "Lab-Script2-Object2"
    set subnet 10.99.99.99 255.255.255.0
  next
end
```

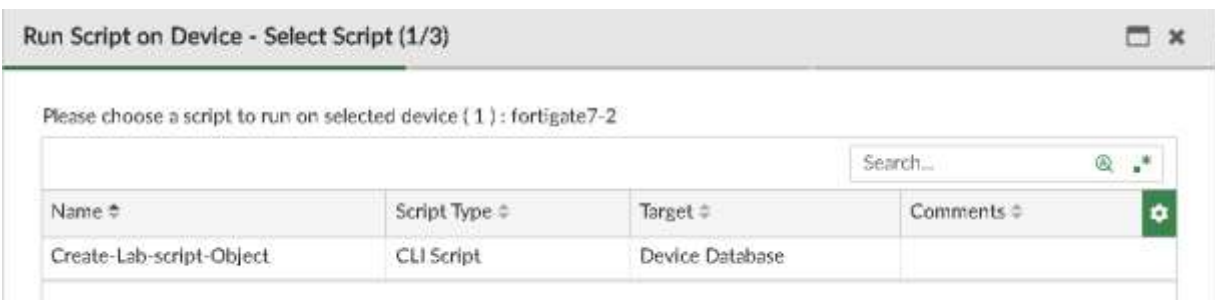


Figure 5 — The same script type, now targeting the Device Database.

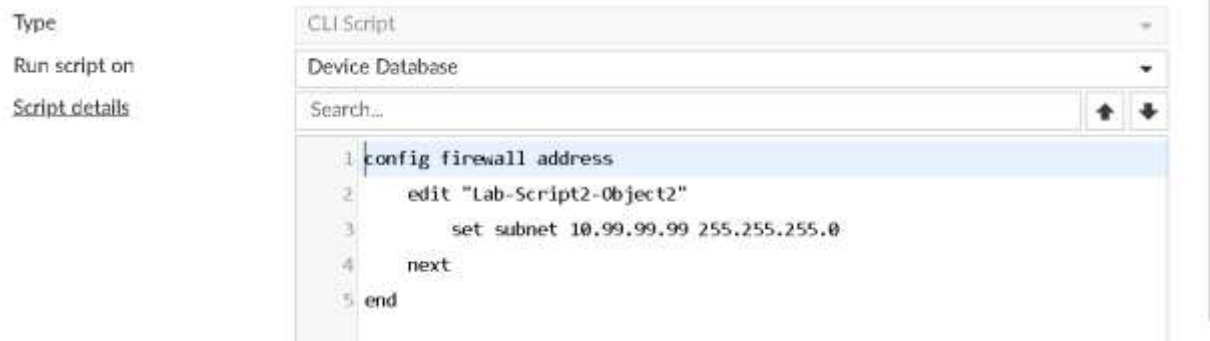


Figure 6 — Script contents with the Device Database target selected.

The script reports success, and nothing appears on the firewall.

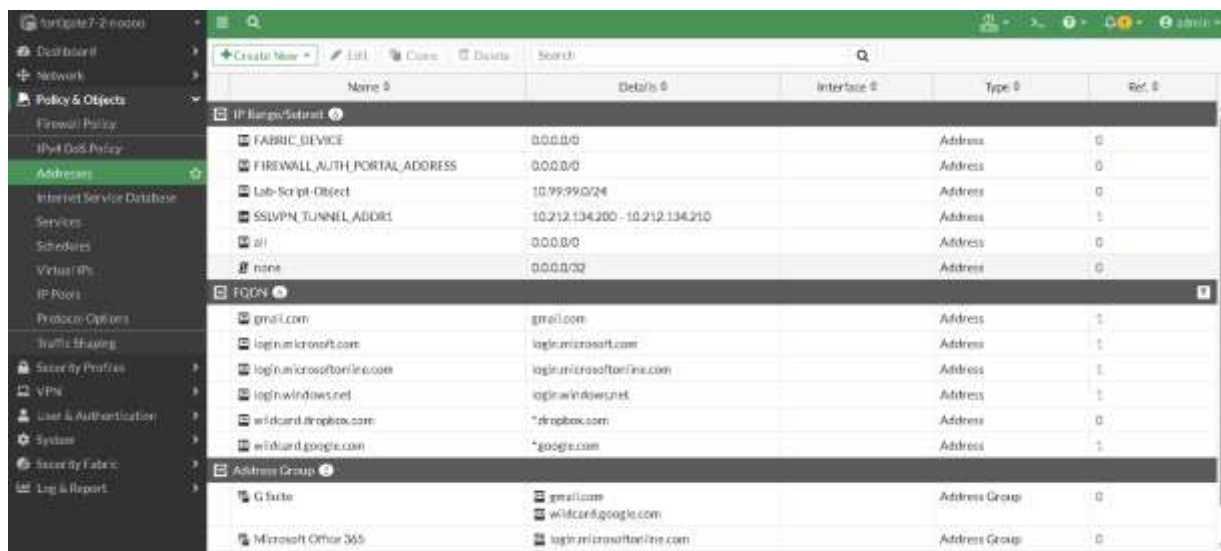


Figure 7 — The FortiGate address list unchanged after the database script ran.

What did change is the configuration status. The device goes to **Modified**, exactly as it did after the manual database edit in Lab 5 — because a database script is simply another way of editing the database.

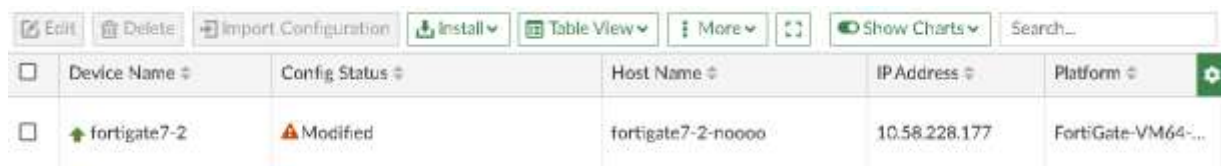


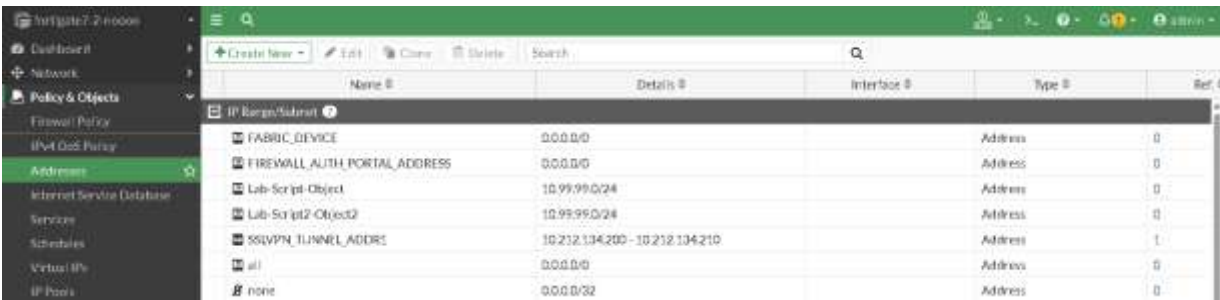
Figure 8 — Configuration status showing Modified after the database script.

► 1.4 Installing the staged change

An install sends it, and the status returns to Synchronized.

☐	Device Name	Config Status	Host Name	IP Address	Platform	
☐	fortigate7-2	✓ Synchronized	fortigate7-2-noooo	10.58.228.177	FortiGate-VM64...	

Figure 9 — Synchronized after the install.



Name	Details	Interface	Type	Ref
FABRIC_DEVICE	0.0.0.0/0		Address	0
FIREWALL_AUTH_PORTAL_ADDRESS	0.0.0.0/0		Address	0
Lab-Script-Object	10.99.99.0/24		Address	0
Lab-Script2-Object2	10.99.99.0/24		Address	0
SSLVPN_TUNNEL_ADDRES	10.212.134.200 - 10.212.134.210		Address	1
all	0.0.0.0/0		Address	0
none	0.0.0.0/32		Address	0

Figure 10 — Both objects now present on the FortiGate — the direct one and the installed one.

The difference between the two targets

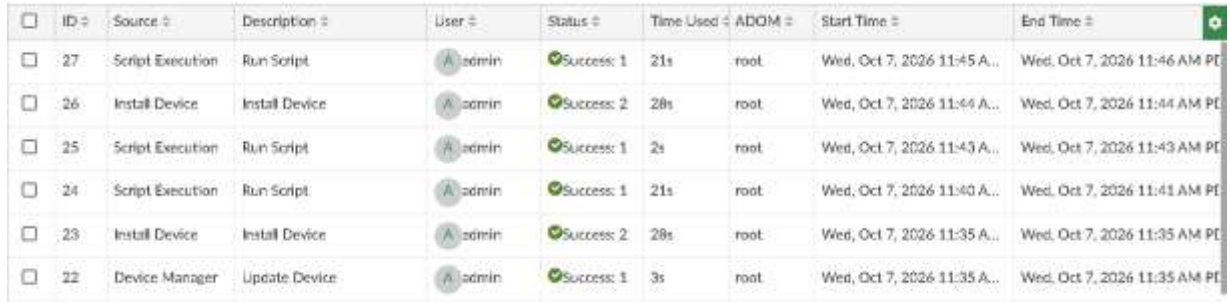
A script run against the remote FortiGate changes the firewall and leaves the configuration status alone. A script run against the device database changes nothing on the firewall and leaves the device Modified until it is installed.

The database target is the safer default. The change can be previewed as CLI before it is sent and is recorded as a revision; the direct target bypasses both, and also leaves FortiManager's stored copy behind until the next retrieve.

Observation	The second object was written as set subnet 10.99.99.99 255.255.255.0 and appears in the address list as 10.99.99.0/24.
Why	FortiOS normalises a host address against its mask to the network address. The value was accepted rather than rejected, which is the part worth noticing.
Risk	A script that silently normalises is a script whose stored result is not what the author typed. Objects created in bulk this way need verifying against the device, not against the script.

▸ 1.5 Script execution is audited

Each run is recorded in Task Monitor as a Script Execution task, interleaved with the Install Device tasks it sits between.



ID	Source	Description	User	Status	Time Used	ADOM	Start Time	End Time
27	Script Execution	Run Script	admin	Success: 1	21s	root	Wed, Oct 7, 2026 11:45 A...	Wed, Oct 7, 2026 11:46 AM PT
26	Install Device	Install Device	admin	Success: 2	28s	root	Wed, Oct 7, 2026 11:44 A...	Wed, Oct 7, 2026 11:44 AM PT
25	Script Execution	Run Script	admin	Success: 1	2s	root	Wed, Oct 7, 2026 11:43 A...	Wed, Oct 7, 2026 11:43 AM PT
24	Script Execution	Run Script	admin	Success: 1	21s	root	Wed, Oct 7, 2026 11:40 A...	Wed, Oct 7, 2026 11:41 AM PT
23	Install Device	Install Device	admin	Success: 2	28s	root	Wed, Oct 7, 2026 11:35 A...	Wed, Oct 7, 2026 11:35 AM PT
22	Device Manager	Update Device	admin	Success: 1	3s	root	Wed, Oct 7, 2026 11:35 A...	Wed, Oct 7, 2026 11:35 AM PT

Figure 11 — Task Monitor — Script Execution and Install Device entries from this section.

The sequence is readable from the log alone: run script, run script, install, run script, install. On an appliance where several administrators use scripts, this is the only record of who ran what against which device.

02. Provisioning Templates

A script is a block of CLI somebody wrote. A provisioning template is a structured form for one area of device configuration — DNS, SNMP, static routes, interfaces — that FortiManager renders into CLI itself and keeps attached to the devices it is assigned to.

The distinction that matters is persistence. A script is run once and forgotten; a template stays assigned, so a device that drifts is brought back in line at the next install and a new device inherits the configuration as soon as it joins.

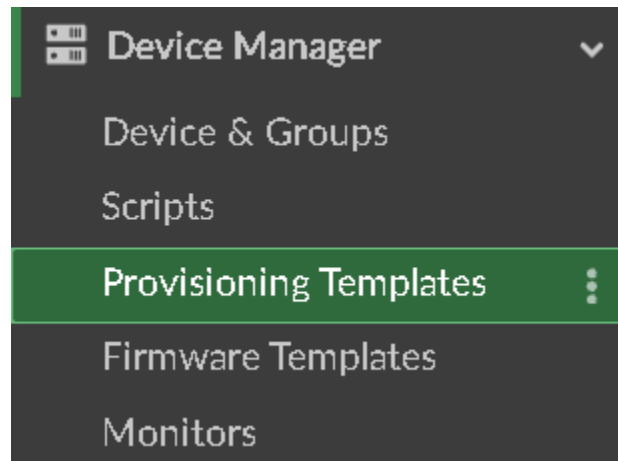


Figure 12 — Device Manager > Provisioning Templates.

► 2.1 Creating a system template

A blank system template was created with only the DNS section enabled, to keep the resulting change small enough to read in a preview.

Figure 13 — Create Blank System Template — DNS enabled, primary 1.1.1.1 and secondary 8.8.8.8.

Before assignment, the FortiGate is still using the DNS servers it had.

```
fortigate7-2-noooo # get system dns
primary           : 96.45.45.45
secondary         : 96.45.46.46
```

Figure 14 — The device's DNS configuration before the template is assigned.

► 2.2 Assigning it

A template does nothing until it is assigned. The context menu carries the two operations that matter — assignment, and a CLI preview of what the template will produce.



Figure 15 — Template context menu — Assign to Device/Group and Preview CLI Configuration.

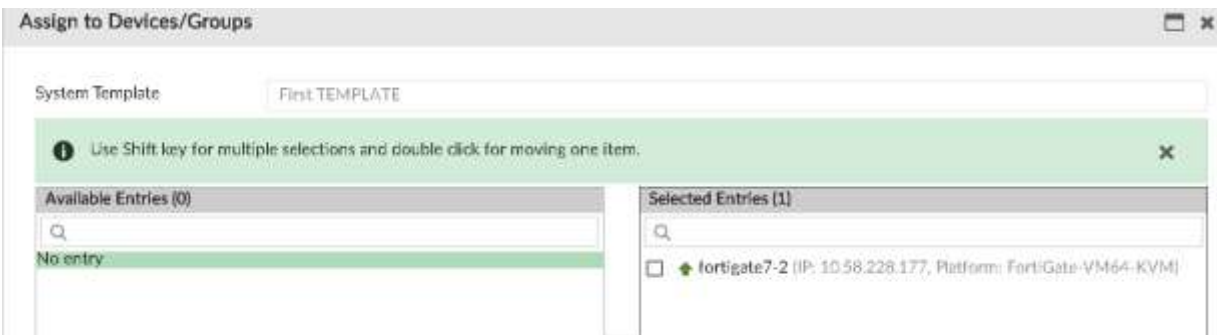


Figure 16 — Assigning the template to fortigate7-2.

Assignment behaves like any other database change: the device goes Modified and waits for an install.

☐	Device Name	Config Status	Host Name	IP Address	Platform	
☐	fortigate7-2	Modified	fortigate7-2-noooo	10.58.228.177	FortiGate-VM64-...	

Figure 17 — Configuration status Modified after assignment.

▸ 2.3 Previewing what it produces

The preview renders the template as the CLI it will send — four lines, and nothing else.



Figure 18 — Install preview of the DNS template.

This is worth reading before the first install of any template. One that enables more sections than intended produces a preview far longer than expected, and the preview is the only place that is visible before the change reaches the firewall.

▸ 2.4 A second template

A static-route template was created alongside it, so that the grouping in the next section has something to group.



Figure 19 — Static route template — default route via 10.58.228.107 on port1.

03. Template Groups

Templates can be bundled so that a device receives a whole configuration baseline from one assignment rather than one assignment per template.

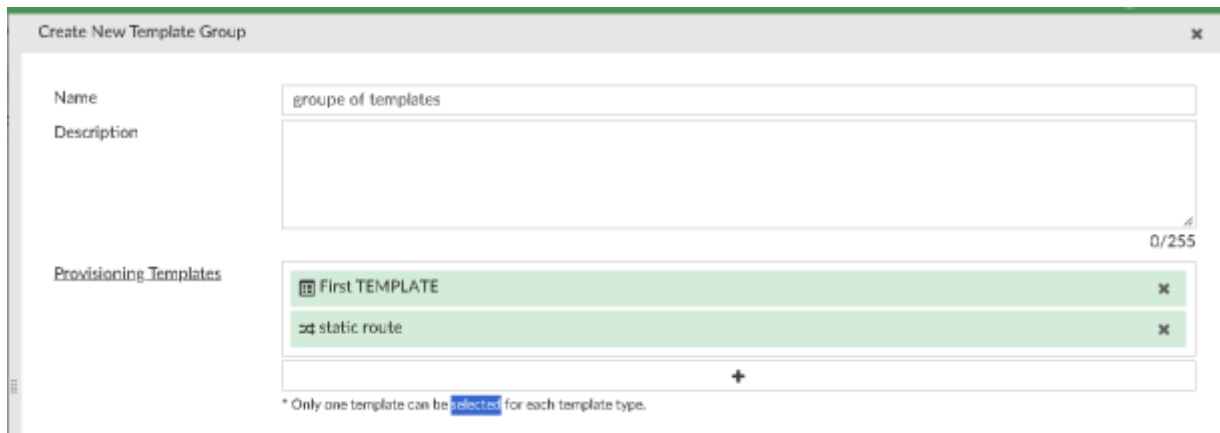


Figure 20 — Create New Template Group combining the system and static-route templates.

One template per type, per group

The dialog states the constraint in small print: **only one template can be selected for each template type**. A group cannot hold two system templates, so overlapping baselines have to be resolved into a single template rather than layered.

This is the practical limit on template groups as a standardization mechanism — the grouping is a bundle, not a merge.



Figure 21 — The template group created, listing both member templates.

04. Meta Fields and Metadata Variables

Two features have near-identical names, sit in different menus, and do different things. Confusing them is the most common reason a variable-driven template quietly fails to work.

Feature	Where it lives	What it is for
Meta Fields	System Settings > Advanced > Meta Fields	Custom attributes attached to devices, ADOMs or objects. Documentation and reporting — somewhere to record a site code or an owner.
Metadata Variables	Policy & Objects > Object Configurations > Advanced > Metadata Variables	Substitutable values, resolved inside scripts and templates at install time, per device.

Only the second kind substitutes. A meta field can be populated on every device in the estate and will still never appear in a configuration.

▸ 4.1 Meta fields

The screenshot shows a 'Create New Meta Fields' dialog box with the following fields and values:

- Object:** Device (selected from a dropdown)
- Name:** samsung (text input)
- Length:** 20 (selected from a dropdown)
- Importance:** Optional (selected), Required (highlighted in green)
- Status:** A toggle switch is turned on (green).

Figure 22 — Create New Meta Fields — object type, name, length and whether it is required.

Setting a meta field to Required is worth understanding before using it: the field then has to be populated on every device of that object type, and devices missing it will block operations until it is filled in.

► 4.2 Metadata variables

A metadata variable carries a default value and, optionally, a different value per device. The default is the fallback; the per-device mapping is the point of the feature.



Figure 23 — Create New Metadata Variables — `site_subnet` with a default of 10.0.0.0.



Figure 24 — Per-device mapping — `fortigate7-2` overriding the default with 10.58.228.0.

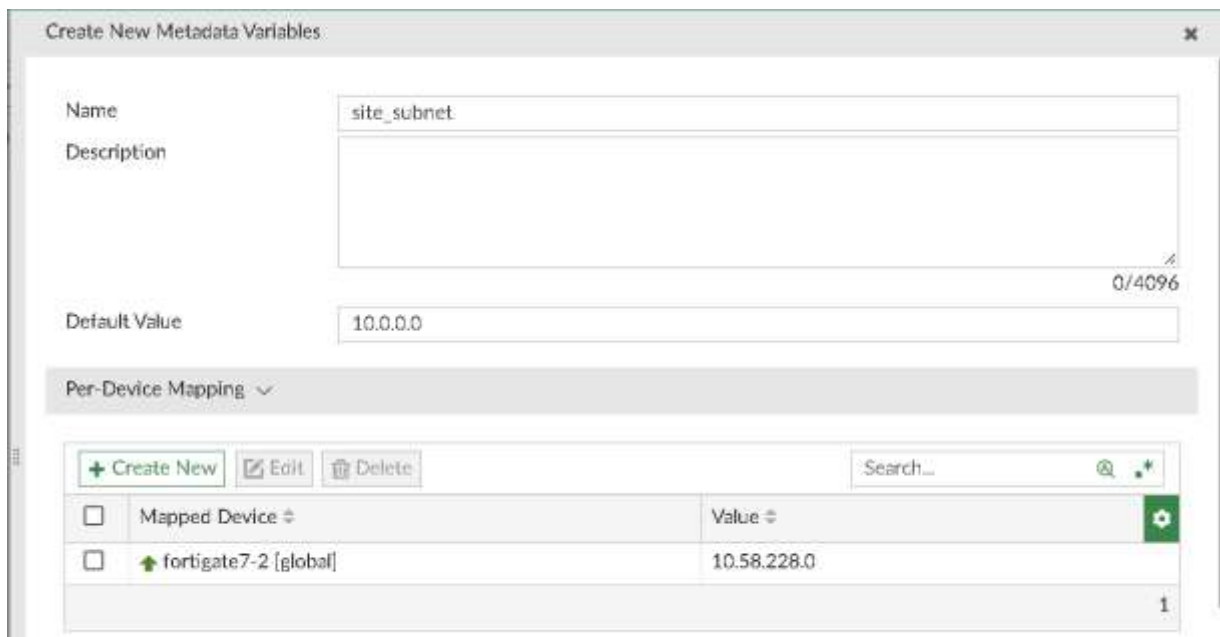


Figure 25 — The completed variable with its default and one device mapping.

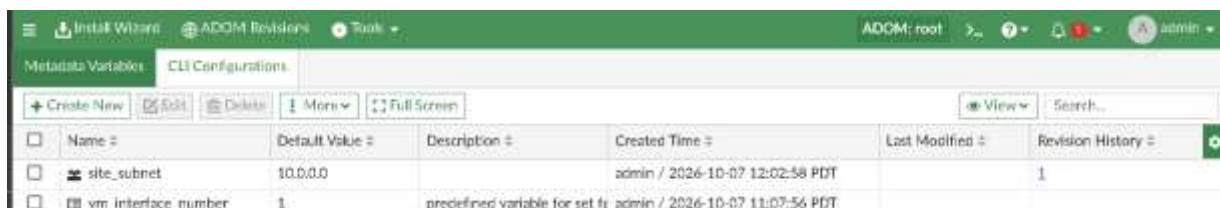


Figure 26 — `site_subnet` listed alongside the predefined `vm_interface_number` variable.

► 4.3 Using the variable

A variable is referenced in a CLI template with `$(name)` syntax. The template below creates one address object whose subnet is whatever the variable resolves to for the device being installed.

```
config firewall address
  edit "Site-LAN"
    set subnet $(site_subnet) 255.255.255.0
  next
end
```

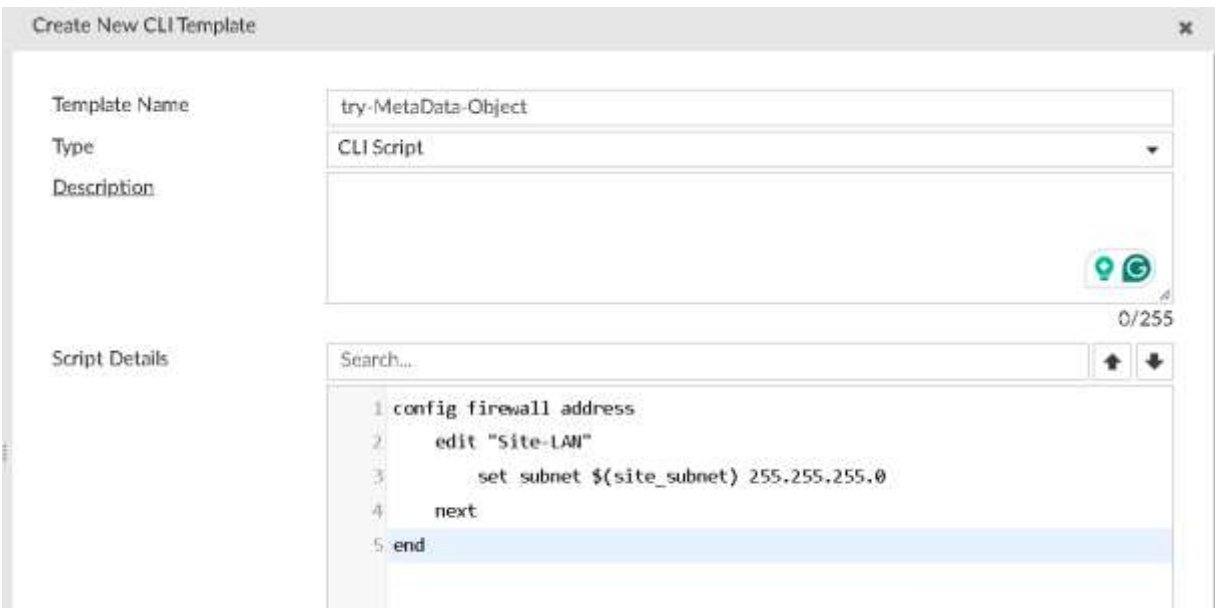


Figure 27 — CLI template referencing `$(site_subnet)`.

► 4.4 Verifying the substitution



Figure 28 — The CLI template assigned to fortigate7-2, with `site_subnet` shown as the variable in use.

✓	Device Name ⇅	Config Status ⇅	Host Name ⇅	IP Address ⇅	Platform ⇅	⚙
✓	fortigate7-2	✓ Synchronized	fortigate7-2-noooo	10.58.228.177	FortiGate-VM64-...	

Figure 29 — Synchronized after installing the template.

The object on the firewall carries the per-device value rather than the default.

```
fortigate7-2-noooo # show firewall address Site-LAN
config firewall address
  edit "Site-LAN"
    set uuid 1f257f5e-c282-51f1-1842-498538b37432
    set subnet 10.58.228.0 255.255.255.0
  next
end
```

Figure 30 — Site-LAN on the FortiGate resolved to 10.58.228.0/24.

One template, one object definition, and a different subnet on every firewall it reaches. Without variables the same result needs one template per site, and the number of templates grows with the estate.

Observation	The installed object carries 10.58.228.0, the per-device mapping, not the 10.0.0.0 default.
Why it matters	An unresolved or misspelled variable does not fail loudly — the default value is installed instead, so the firewall ends up with a plausible-looking object containing the wrong network.
Diagnosis	Verify on the device after the first install of any variable-driven template. The install result and the configuration status both report success either way.